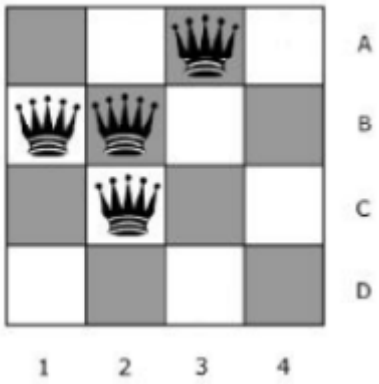
BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Lê Đoàn Kim Ngân
MSSV / Lớp	3123410231 / DCT1236

Bài 1. 4-Queens và Hill Climbing



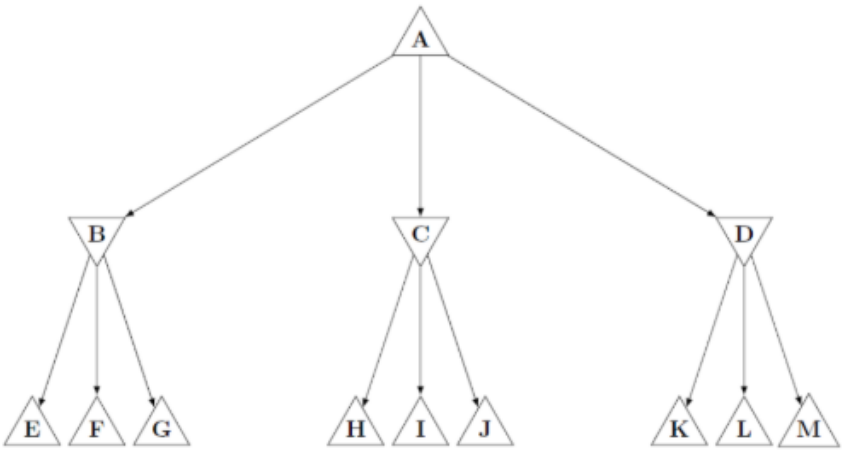
a) Ban đầu có 4 cặp quân hậu xung đột. Sau một vòng lặp của Hill Climbing, quân hậu tại B2 di chuyển đến D4 (đi chéo qua C3), , tạo trạng thái mới A3, B1, C2, D4. Trạng thái mới còn 1 cặp quân hậu xung đột.

Sau 1 vòng lặp: **A3, B1, C2, D4** (còn xung đột B1–C2).

b) Sau khi di chuyển đến D4, số cặp xung đột giảm còn 1. Thuật toán tiếp tục tìm trạng thái tốt hơn, nhưng không có nước di chuyển hợp lệ nào giúp giảm số xung đột xuống 0.

→ **Chọn B. Cực trị cục bộ (local optimum)** (còn 1 xung đột nên không phải global optimum).

Bài 2. Alpha-Beta + Iterative Deepening



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

a) Sắp theo f từ tốt nhất đến xấu nhất cho nút MAX A: D(19) > C(14) > B(4).

Chọn **D → C → B**.

b) Duyệt theo thứ tự D, C, B:

Xét các cây con:

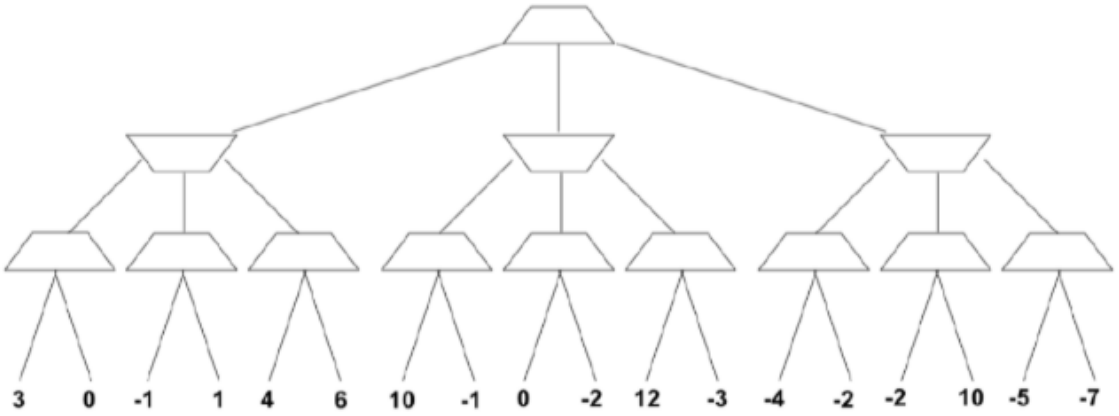
- D: K=23, L=20, M=26 nên $D = min(23,\ 20,\ 26) = 20$, α của A = 20.

- C: H=17 ≤ α=20 nên cắt tỉa, I và J không được thăm (C không thể đem lại giá trị tốt hơn 20 cho A).
- B: E=5 ≤ 20 nên cắt tỉa, F và G không được thăm (B không thể đem lại giá trị tốt hơn 20 cho A).

Các nút không được thăm: **I, J, F, G**. Giá trị tại A là 20, đi vào D.

c) Không. Alpha-Beta Pruning chuẩn chỉ cắt tỉa các nhánh không ảnh hưởng đến kết quả Minimax trên cùng độ sâu tìm kiếm. Trong bài này, ta giới hạn độ sâu và sử dụng hàm đánh giá (f) tại các nút cháu. Vì (f) chỉ là giá trị ước lượng, không phải giá trị Minimax thực tế của toàn bộ cây, nên nước đi được chọn có thể khác với Minimax trên toàn bộ cây.

Bài 3. Alpha-Beta Pruning



Nhánh trái (MIN_1), α/β ban đầu (-∞, +∞):

- MAX (3, 0): (-∞,+∞), trả về 3. MIN_1 có β=3.
- MAX (-1, 1): (-∞,3), trả về 1. MIN_1 có β=1.
- MAX (4, 6): (-∞,1). Lá 4 làm α=4 ≥ β=1 nên X ở lá 6, trả về 4.
- MIN_1 trả về 1, gốc có α=1.

Nhánh giữa (MIN_2), (α,β) = (1,+∞):

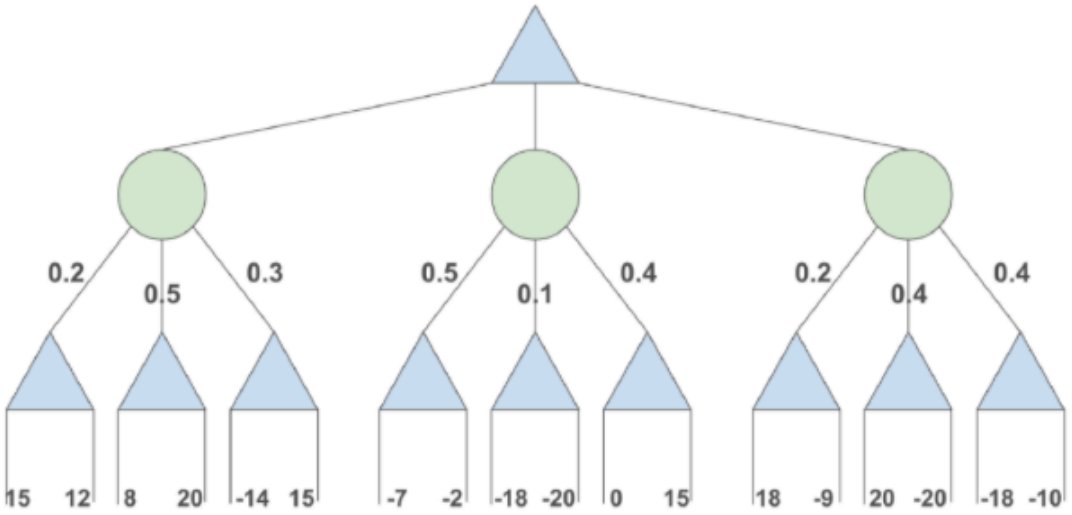
- MAX (10, -1): trả về 10. MIN_2 có β=10.
- MAX (0, -2): (1,10), trả về 0. MIN_2 có v=0 ≤ α=1 nên X cả MAX (12, -3).
- MIN_2 trả về 0.

Nhánh phải (MIN_3), (α,β) = (1,+∞):

- MAX (-4, -2): trả về -2. MIN_3 có v= -2 ≤ α=1 nên X cả hai MAX còn lại: (-2,10) và (-5,-7).
- MIN_3 trả về -2.

Kết quả: giá trị Minimax tại gốc = max(1, 0, -2) = **1**. Nước đi tối ưu của MAX là **nhánh trái**.

Bài 4. Expectimax và Chance Nodes



a)

Chance node	Giá trị các MAX con	Kỳ vọng
Trái	15, 20, 15	0.2·15 + 0.5·20 + 0.3·15 = 17.5
Giữa	-2, -18, 15	0.5·(-2) + 0.1·(-18) + 0.4·15 = 3.2
Phải	18, 20, -10	0.2·18 + 0.4·20 + 0.4·(-10) = 7.6

Gốc = max(17.5, 3.2, 7.6) = **17.5**, đi nhánh trái.

b) Không nút nào cắt tỉa được. Chance node là trung bình có trọng số, nên một con chưa xem với giá trị lớn hoặc nhỏ tùy ý có thể làm đổi hẳn kỳ vọng. Khi lá không bị chặn thì không có cận nào để so với giá trị tốt nhất hiện tại.

c) Với lá trong [-20, 20]:

- **Chance trái:** Tính đầy đủ giá trị kỳ vọng:
  ($V_L = 0.2(15) + 0.5(20) + 0.3(15) = 17.5$).
  Gốc MAX có giá trị tốt nhất hiện tại là 17.5.
- **Chance giữa:** Sau khi nút MAX đầu tiên trả về -2, cận trên của chance node là:
  ($U_M = 0.5(-2) + 0.1(20) + 0.4(20) = 9 < 17.5$).
  Vì cận trên nhỏ hơn giá trị tốt nhất hiện tại ở gốc, ta cắt tỉa hai nhánh MAX còn lại: (-18, -20) và (0, 15). Nếu tính đầy đủ, giá trị kỳ vọng của chance giữa là:
  ($V_M = 0.5(-2) + 0.1(-18) + 0.4(15) = 3.2$).
- **Chance phải:** Sau khi xét hai nút MAX đầu tiên có giá trị 18 và 20, cận trên là: ($U_R = 0.2(18) + 0.4(20) + 0.4(20) = 19.6 > 17.5$).
  Vì cận trên vẫn lớn hơn giá trị tốt nhất hiện tại ở gốc, ta chưa thể cắt tỉa nhánh cuối. Xét nút MAX cuối, nhận giá trị -10. Giá trị kỳ vọng của chance phải là: ($V_R = 0.2(18) + 0.4(20) + 0.4(-10) = 7.6$).

Vậy giá trị tại gốc là: $V(A) = \max(17.5, 3.2, 7.6) = 17.5$

Kết luận: MAX chọn nhánh trái.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking (FC) khác AC-3 ở ba điểm:

- **Phạm vi lan truyền:** FC chỉ kiểm tra các biến kề với biến vừa gán, xóa giá trị xung đột trong miền của chúng và không tiếp tục lan truyền. AC-3 lan truyền theo hàng đợi: miền nào bị thu hẹp thì kiểm tra lại các cung liên quan, nên ảnh hưởng đi qua cả đồ thị ràng buộc.
- **Thời điểm:** FC chạy sau mỗi phép gán trong lúc backtracking. AC-3 có thể chạy trước khi tìm kiếm (tiền xử lý) hoặc sau mỗi phép gán (MAC).
- **Chi phí:** FC rẻ (mỗi phép gán chỉ xét các biến kề). AC-3 tốn hơn (worst case $O(e \cdot d^3)$ với ràng buộc nhị phân) nhưng cắt được nhiều giá trị hơn và phát hiện dead end sớm hơn.

b) Vì sao sử dụng MRV và LCV?

- **MRV:** chọn biến có ít giá trị hợp lệ nhất (fail-first). Biến này dễ gây thất bại nhất nên nếu có lỗi thì lộ ra sớm và cắt được cả nhánh lớn. Biến chỉ còn 1 giá trị thì gán luôn.
- **LCV:** chọn giá trị loại bỏ ít giá trị nhất trong miền các biến kề. Nó giữ nhiều lựa chọn cho các biến sau, nên dễ tìm ra nghiệm mà ít phải backtrack.

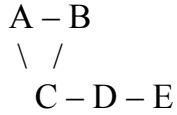
Bài 6. CSP – Điều phối không lưu

a) Mô hình hóa

- Biến: A, B, C, D, E. Mỗi biến X = (t_X, r_X) với t là khung giờ, r là đường băng.
- Miền: {1,2,3,4} × {QT, ND}.
- Unary: t_B = 1; t_D ≥ 3; t_A ≤ 2.
- Binary: t_D < t_C, và với mọi X ≠ Y: (t_X, r_X) ≠ (t_Y, r_Y).

b) Miền mới của mỗi biến chỉ còn khung giờ: {1,2,3,4}. A, B, C dùng đường băng QT nên t_A ≠ t_B, t_A ≠ t_C, t_B ≠ t_C. D, E dùng đường băng ND nên t_D ≠ t_E. Thêm t_D < t_C.

Đồ thị ràng buộc có 5 cạnh: A–B, A–C, B–C, C–D, D–E.



c) Arc consistency

- Unary: B={1}, D={3,4}, A={1,2}.
- A≠B nên bỏ 1 khỏi A, được A={2}.
- B≠C nên bỏ 1 khỏi C. A≠C nên bỏ 2 khỏi C, được C={3,4}.
- D<C: D=4 không có giá trị nào của C lớn hơn nên bỏ 4, được D={3}. Rồi C>3 nên C={4}.
- D≠E nên bỏ 3 khỏi E, được E={1,2,4}.

Miền cuối: **A={2}, B={1}, C={4}, D={3}, E={1,2,4}**.

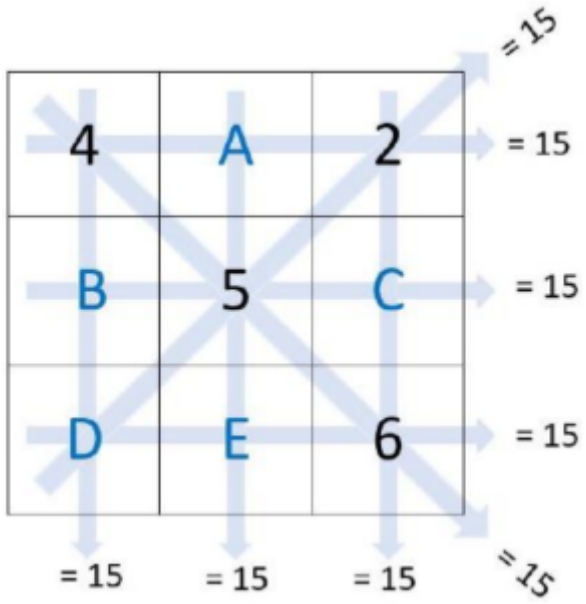
d) FC + Backtracking (MRV). Sau unary: A{1,2}, B{1}, C{1-4}, D{3,4}, E{1-4}.

- B=1 (MRV, miền 1 giá trị). FC: A{2}, C{2,3,4}.
- A=2. FC: C{3,4}.
- C và D hòa (cùng 2 giá trị), chọn chữ nhỏ hơn là C. C=3. FC: D<3 nên D rỗng, dead end, hoàn tác C=3.

- C=4. FC: D{3}.
- D=3. FC: E{1,2,4}.
- E=1. Xong.

Thứ tự: **(B,1), (A,2), (C,3) ✗, (C,4), (D,3), (E,1)**. Nghiệm: A=2, B=1, C=4, D=3, E=1.

Bài 7. CSP – Magic Square



a) Biến: A, B, C, D, E. Miền: {1,3,7,8,9} (các số còn lại sau khi trừ 2, 4, 5, 6 đã dùng).

- AllDiff(A,B,C,D,E).
- Unary: hàng 1 → A=9; cột 3 → C=7; đường chéo 2+5+D → D=8.
- Binary: B+C=10 (hàng 2), D+E=9 (hàng 3), B+D=11 (cột 1), A+E=10 (cột 2).
- Đường chéo chính 4+5+6=15 đã thỏa sẵn.

b) Bỏ qua ràng buộc AllDiff, ta có các ràng buộc giữa từng cặp biến:

- $B + C = 10$: cạnh B–C.
- $B + D = 11$: cạnh B–D.
- $D + E = 9$: cạnh D–E.
- $A + E = 10$: cạnh A–E.

Đồ thị ràng buộc: C – B – D – E – A

c) AC-3

- Unary: A={9}, C={7}, D={8}.
- B+C=10 nên B={3}. B+D=11 khớp (3+8).
- D+E=9 nên E={1}. A+E=10 khớp (9+1).

Mọi miền đều còn 1 giá trị nên có nghiệm ngay: **A=9, B=3, C=7, D=8, E=1**.

4	9	2
3	5	7
8	1	6

Các hàng, cột và hai đường chéo đều bằng 15.

Bài 8. MRV và LCV trong thực hành

a) MRV + LCV

- MRV chọn Ellyse (2 màu). LCV: Đỏ loại 2 giá trị (của AJ và Sunny), Hồng chỉ loại 1 (của Sunny) nên chọn Ellyse = Hồng.
- Sunny còn {Đỏ, Vàng} (2 màu) nên chọn Sunny. Hai màu loại như nhau, hòa nên theo thứ tự cầu vồng: Sunny = Đỏ.
- AJ còn {Vàng, Xanh lá}. Hòa, theo cầu vồng: AJ = Vàng.

Thứ tự gán: Ellyse=Hồng → Sunny=Đỏ → AJ=Vàng. **Số lần backtrack: 0.**

b) Maximum Remaining Values + Most Constraining Value

- AJ và Sunny cùng 3 màu, phá hòa theo thứ tự đề bài liệt kê nên chọn AJ. Most Constraining chọn Đỏ (loại 2 giá trị), nên AJ = Đỏ. Lúc này Sunny{Vàng,Hồng}, Ellyse{Hồng}.
- Max Remaining chọn Sunny (2 màu). Hồng loại Hồng của Ellyse (nhiều ràng buộc hơn Vàng) nên Sunny = Hồng, làm Ellyse hết màu, dead end, backtrack 1 lần.
- Thử Sunny = Vàng, rồi Ellyse = Hồng. Xong.

Số lần backtrack: 1. Nhiều hơn câu (a) vì chiến lược ngược lại làm sai việc: chọn biến dễ nhất trước và chọn giá trị làm khó các biến còn lại nhất. Lỗi được nhận ra muộn nên mất công quay lui, còn MRV/LCV tránh được điều đó ngay từ đầu.